

DAY-2 TASK

WEBPAGE AND WEBSITE

Yes. The easiest way to understand **website vs webpage** is to imagine a **book**.

- **Website = the entire book**
- **Webpage = one page of that book**
- **Web browser = the thing you use to read the book**
- **Internet = the huge library containing many books**

Let's understand it from the beginning.

1. What is a Webpage?

A **webpage** is a single document/page that you can view on the internet using a web browser such as Chrome, Edge, Firefox, or Safari.

For example, when you open a page that says:

"Welcome to Amazon"

that particular page is a **webpage**.

Another page might show:

"Today's Deals"

That is another webpage.

Another might show:

"Your Orders"

That is also a webpage.

So:

One webpage = one individual page of information on the web.

Simple example

Imagine a school textbook:

Book

|

|— Page 1

|— Page 2

|— Page 3

|— Page 4

└ Page 5

A website works similarly:

Website

|

|— Home webpage

|— About webpage

|— Products webpage

|— Contact webpage

└ Login webpage

The individual pages are **webpages**.

2. What is a Website?

A **website** is a collection of related webpages and other resources that are organized under one common domain name.

For example, imagine a company's website:

www.example.com

It might contain:

www.example.com/

↓

Home

www.example.com/about

↓

About Us

www.example.com/products



Products

www.example.com/contact



Contact Us

All these webpages together form the **website**.

So the basic relationship is:

Many webpages → make up a website.

But technically, a website can also contain things other than HTML webpages, such as images, videos, CSS files, JavaScript files, fonts, PDFs, and other resources.

3. Website vs Webpage

Website	Webpage
A collection of related webpages	A single page/document
Usually has multiple pages	Is one individual page
Has a domain name	Has a specific URL
Example: Amazon's entire site	Example: Amazon's product page
Like an entire book	Like one page of a book

Very simple example

Think about Wikipedia.

Wikipedia is a **website**.

Inside Wikipedia, you can have individual webpages such as:

- India
- HTML
- JavaScript
- Chennai

- Computer Science

Each individual article page is a **webpage**.

4. But how was the first webpage actually made?

This is where things become interesting.

A webpage is fundamentally a **document**.

When the web was first developed, a webpage was primarily a document containing **text and links to other documents**.

The first website and webpage were created by **Tim Berners-Lee** at CERN around 1990–1991.

The important idea was:

"What if documents stored on different computers could be connected to each other through links?"

That idea became the **World Wide Web**.

5. Before webpages existed

Let's go back further.

Suppose you had a computer containing this information:

Computer A

Information about Physics

Information about Chemistry

Information about Mathematics

Someone on another computer couldn't simply click a link and open that information through a worldwide system.

There were computer networks, but the **World Wide Web** as we know it didn't exist yet.

Tim Berners-Lee proposed a system where documents could be connected using hyperlinks.

For example:

Document A

|
| hyperlink

↓

Document B

|
| hyperlink

↓

Document C

That is the fundamental idea behind the web.

6. The first webpage

The first webpage was created by **Tim Berners-Lee** at CERN.

It wasn't like today's websites.

Today's websites can have:

- animations
- videos
- login systems
- shopping carts
- advertisements
- interactive buttons
- complex graphics
- databases
- artificial intelligence

The early web was much simpler.

The webpage mainly explained things such as:

- What the World Wide Web was
- How the web worked

- How people could create webpages
- How to access information
- Links to other information

So the original idea of a webpage was essentially:

A document available through a network that can contain hyperlinks to other documents.

7. How was that webpage created?

This brings us to **HTML**.

HTML stands for:

HyperText Markup Language

Let's break that down.

HyperText

"Hyper" means something that goes beyond the normal/linear flow.

"Text" means written information.

So **hypertext** is text that can connect you to other information.

For example:

Learn HTML

You click "Learn HTML" and another document opens.

That connection is a **hyperlink**.

So:

Document A

|
| hyperlink

↓

Document B

This is the "hyper" part of **hypertext**.

8. What does Markup Language mean?

This is extremely important for understanding HTML.

A **markup language** is a language used to describe the structure or meaning of content.

Imagine you have this plain text:

My Website

Welcome to my website.

This is my first paragraph.

A computer needs to understand:

- Which part is the title?
- Which part is a heading?
- Which part is a paragraph?
- Which part is a link?
- Which part is an image?

HTML provides that structure.

Conceptually:

Title

↓

Heading

↓

Paragraph

↓

Link

HTML tells the browser **what each piece of content is**.

That's why HTML is called a **markup language**, rather than a programming language.

9. How a webpage is built

A modern webpage usually involves several technologies working together.

Think of building a house.

HTML = structure

HTML tells the browser:

"This is a heading."

"This is a paragraph."

"This is an image."

"This is a button."

"This is a link."

So HTML is like the **skeleton/structure of the webpage**.

CSS = appearance

CSS tells the browser things such as:

Make this heading large.

Make the background this color.

Put this section on the left.

Add spacing.

Make the page responsive.

So CSS is like the **appearance/design** of the webpage.

JavaScript = behavior

JavaScript tells the browser what should happen when the user interacts with the page.

For example:

When the user clicks this button, do something.

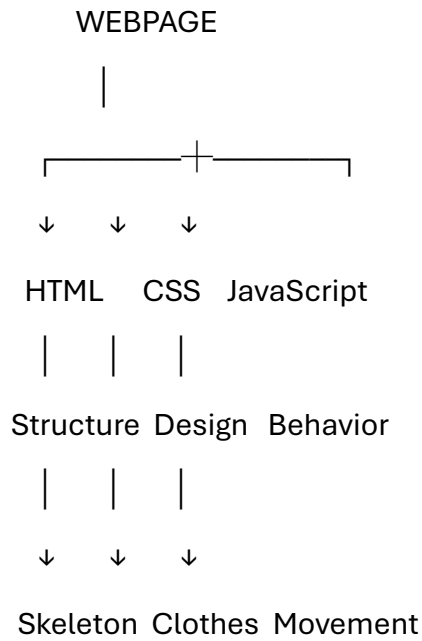
When the user submits this form, validate it.

When the user opens this menu, show the menu.

So JavaScript provides **behavior and interactivity**.

10. Think of a webpage as a human body

This is one of the easiest ways to remember it.



For example, imagine a login page.

HTML

Creates:

- Login heading
- Username field
- Password field
- Login button

CSS

Controls:

- Colors
- Font
- Size
- Position
- Spacing
- Layout

JavaScript

Controls:

- What happens when Login is clicked

- Form validation
- Showing error messages
- Sending information to a server

Together they produce the webpage the user sees.

11. What happens when you type a website address?

Suppose you type:

www.example.com

into Chrome.

A simplified process is:

You

|

| type website address

↓

Browser

|

| requests information

↓

Internet

|

↓

Web Server

|

| sends webpage/resources

↓

Browser

|

| interprets HTML + CSS + JavaScript

↓

Screen

|

↓

Webpage appears

This is a very important concept.

The **website is stored/served from computers called servers**, and your browser requests the resources needed to display a page.

12. Website, webpage, browser, server, internet — don't confuse them

These five terms are commonly confused.

Internet

The enormous worldwide network connecting computers and devices.

World Wide Web

A system built on the internet that allows documents/resources to be accessed through links and URLs.

Website

A collection of related web resources/pages available under a domain.

Webpage

One individual document/page within that website.

Web browser

Software such as Chrome, Edge, Firefox, or Safari that retrieves and displays webpages.

Web server

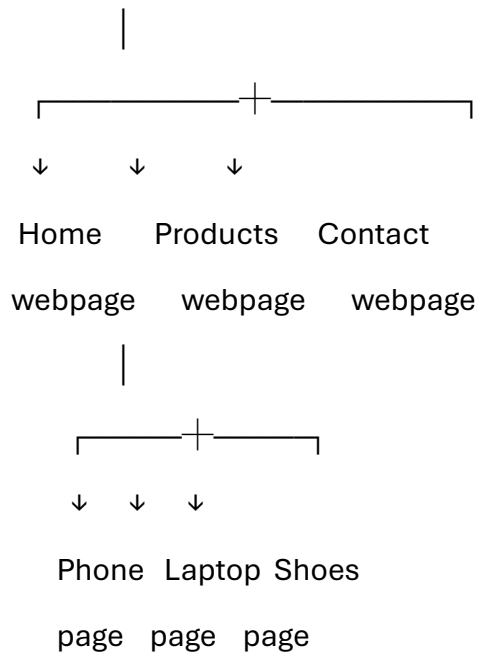
A computer/system that stores or serves website resources when requested.

13. A complete real-world example

Imagine a shopping website.

WEBSITE

example-shop.com



The entire thing is the **website**.

Each individual page is a **webpage**.

14. Is every URL a webpage?

Not necessarily.

A URL can point to many types of resources.

For example:

<https://example.com/about>

might point to a webpage.

But a URL could also point to:

image.jpg

or:

document.pdf

or another type of resource.

So **URL** and **webpage** aren't the same thing.

A URL is essentially an **address used to locate a resource**.

15. Why do we call it a "website"?

Think about the word:

Web + Site

"Site" means a location/place.

So a **website** is essentially a place on the World Wide Web where related information/resources are made available.

Similarly:

Website



Location on the web



Contains webpages/resources

16. The whole evolution

You can understand the history roughly like this:

Computers



Computer Networks



Internet



World Wide Web



Web Documents



HTML



Webpages



Websites



CSS + JavaScript



Interactive Web Applications



Modern Web

The web started much more simply than the applications we use today.

Early webpages were primarily **documents connected by hyperlinks**.

Over time, technologies such as HTML, CSS, JavaScript, databases, servers, APIs, and frameworks made websites much more interactive.

17. One important distinction: Website vs Web Application

This will become important when you learn React and JavaScript.

A traditional website might mainly provide information:

Home

About

Services

Contact

A **web application** behaves more like software running in your browser.

Examples include:

- Online banking
- Gmail
- Google Docs
- Online shopping systems
- Social media platforms

They still use webpages and web technologies, but they provide much more interaction and functionality.

So you can think of the progression as:

Simple document



Webpage



Website



Interactive website



Web application

The easiest definition to remember

A webpage is a single document/page available on the World Wide Web.

A website is a collection of related webpages and other web resources organized under a common domain.

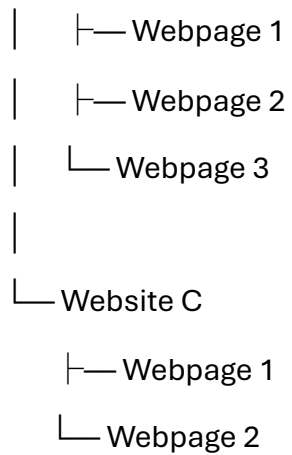
And historically:

The World Wide Web was created by Tim Berners-Lee, and the early web was based on documents written in HTML and connected using hyperlinks.

One final mental picture

INTERNET





Website = collection

Webpage = individual page

HTML = structure of the webpage

CSS = appearance of the webpage

JavaScript = behavior of the webpage

Browser = displays the webpage

Server = provides the webpage/resources

URL = address used to locate a resource

That foundation will make **HTML → CSS → Bootstrap/Tailwind → JavaScript → React → SQL** much easier to understand.